Siddharth Bhat

siddharth.bhat@cl.cam.ac.uk · +44 7442 785227 · pixel-druid.com · Churchill College, Cambridge, UK
github.com/bollu · linkedin.com/in/siddharth-bhat · Google Scholar · UK Global Talent visa (Exceptional Promise)

Summary

ML Performance Engineer at **XTX Markets**; PhD researcher (Cambridge) in formally verified decision procedures and mechanical theorem proving in Lean, with a focus on compiler verification. Co-developer of `bv_decide`, Lean's verified-bitblasting bitvector solver (**5th at SMT-COMP 2025**, `QF_BV` division), and author of the first sound-and-complete, machine-checked decision procedures for *parametric* (width-independent) bitvector theories. **#2 contributor to Lean's bitvector theory** and a top-20 contributor to the prover itself; coauthor of the *Lean 4 Metaprogramming Book*. Led `lean-mlir`, a formal semantics for MLIR in Lean that machine-checks real LLVM peephole rewrites at arbitrary bitwidth. Fluent in Lean, Rocq, F*, C++, Haskell, Rust.

Expertise

- **SMT & decision procedures (`QF_BV`):** co-developed `bv_decide` (verified bitblasting, AIG + CaDiCaL certificate checking) — 5th at SMT-COMP 2025; published sound-and-complete procedures for multi-width parametric bitvectors via principled reductions; current work on mechanized floating-point bitblasting.
- **Compiler verification:** `lean-mlir` — denotational semantics for MLIR dialects in Lean 4, with verified peephole rewriting over SSA regions (ITP 2024); verified dominance and side-effect modelling for MLIR; collaboration with VE-LLVM on Rocq semantics for LLVM; `PolyIR`, a formal specification of polyhedral programs (ETH Zurich).
- **Proof engineering & automation:** tactics and elaboration in Lean 4 (author of the tactics and embedded-DSL chapters of the Lean 4 Metaprogramming Book); tactics for deciding memory non-interference in a Lean-based ARM symbolic simulator (AWS Automated Reasoning Group); retrieval-augmented proof synthesis for F* (ICSE 2025 best paper).

Selected Publications

1. Towards Mechanized Floating Point Bitblasting. *Siddharth Bhat*, Abdalrhman Mohamed, Tobias Grosser. *POPL 2027 (under review)*.
2. Sound and Complete Solving for Multi-Width Parametric Bitvectors via Principled Reductions. *Siddharth Bhat*, Léo Stefanesco, Tobias Grosser. *OOPSLA 2026 (accepted, minor revisions)*.
3. Certified Decision Procedures for Width-Independent Bitvector Predicates. *Siddharth Bhat*, Léo Stefanesco, Chris Hughes, Tobias Grosser. *OOPSLA 2025*.
4. Interactive Bit Vector Reasoning using Verified Bitblasting. Henrik Böving, *Siddharth Bhat*, Alex Keizer, Luisa Cicolini, Leon Frenot, Abdalrhman Mohamed, Léo Stefanesco, Harun Khan, Josh Clune, Clark Barrett, Tobias Grosser. *OOPSLA 2025*.
5. Verifying Peephole Rewriting in SSA Compiler IRs. *Siddharth Bhat*, Alex Keizer, Chris Hughes, Andres Goens, Tobias Grosser. *ITP 2024*.
6. Towards Neural Synthesis for SMT-Assisted Proof-Oriented Programming. Saikat Chakraborty, Gabriel Ebner, *Siddharth Bhat*, Sarah Fakhoury, Shuvendu Lahiri, Nikhil Swamy. *ICSE 2025 (best paper)*.
7. Rewriting Optimization Problems into Disciplined Convex Programming Form. Ramon Fernandez Mir, *Siddharth Bhat*, Andres Goens, Tobias Grosser. *CICM 2024*.

Full list (14 papers, incl. POPL, CGO, ICML, NeurIPS, SC) on Google Scholar.

Software

Lean 4: co-developed the verified bitblasting theory behind `bv_decide`; #2 contributor to the bitvector library; primary author of Lean's LLVM backend.
lean-mlir: formal semantics for MLIR in Lean 4, with machine-checked peephole rewriting over SSA regions.
Lean 4 Metaprogramming Book: authored the chapters on tactics and on metaprogramming for embedded DSLs.
Rocq / VE-LLVM: issues, bug fixes and developer documentation for Rocq; collaboration on VE-LLVM's formal semantics of the LLVM toolchain.
PolyIR: a formal specification of polyhedral programs, for verifying loop transformations.

Experience

2026–present XTX Markets, London — *ML Performance Engineer*. Performance engineering for machine-learning systems.
Sep–Nov '24 Amazon Web Services, Automated Reasoning Group, Austin — *Research Intern*. Tactics for proving memory non-interference in `lnsym`, a Lean-based ARM symbolic simulator.
Jul–Sep '23 Microsoft Research, Redmond — *Research Intern*. Retrieval-augmented theorem proving for the F* proof assistant; ICSE 2025 best paper.
Summer '18 ETH Zurich, Switzerland — *Research Intern*. Formal verification of polyhedral compilation (`PolyIR`).
May–Jul '19 Tweag.io, Paris — *Research Intern*. Re-implemented portions of the GHC runtime for Asterius, a Haskell→WebAssembly compiler.
Mar–Dec '17 ETH Zurich, Scalable Parallel Computing — *Research Intern*. GPU code generation in Polly, LLVM's polyhedral loop optimizer (#3 contributor overall).

Selected Talks

How to trust your peephole rewrites: automatically verifying them for arbitrary width (EuroLLVM '25) · lean-mlir: a workbench for formally verifying peephole optimizations in MLIR (LLVM Dev '24) · (Correctly) extending dominance to MLIR regions (LLVM Dev '23) · MLIR side-effect modelling (LLVM Dev '23).

Education & Awards

PhD, Computer Science, University of Cambridge; transferred from University of Edinburgh — 2022–2026 (writing up)
MS / BTech, Computer Science, IIIT Hyderabad, India — 2015–2021

Awards: 5th place, SMT-COMP 2025 (`QF_BV`) · Best paper, ICSE 2025 · Two Renaissance Philanthropy AI for Maths grants (1 of 30 funded from 280+ applicants; tactic synthesis via Lean metaprogramming)